CIS520
Project 4
Group 8

Zeke Flippo
Vixi Spellman
TJ Tiede

Problem Statement

Given an ASCII encoded file, our goal is write a program that for each line of the input (in order) prints a corresponding line to the output “{line_number}: {max_char}” where max_char is the greatest byte on the line (not including the newline character).

High Level Architecture

Each of our three programs share the same high level architecture. Their differences are discussed in the following section. The fundamental idea is that we minimize the amount of synchronization needed by devising a stratgy that allows each thread to operate as independently as possible.

The main thread is only responsible for:
1. Reading fixed sized batches from the input file. We read the file in batches so that we can still make progress with arbitrarily large inputs while requiring only a fixed amount of memory.
2. Making the current batch available to the worker threads
3. Receiving the worker's results and printing them in order while keeping track of how many lines have been processed
4. Maintenance of invariant 1. (see below).

We partition the batch into sections of size

$$\texttt{bytes\_per\_thread} = \left\lceil \frac{\texttt{batch\_size}}{\texttt{num\_workers}} \right\rceil \quad \text{where } \texttt{'num\_workers} = \texttt{num\_threads} - 1.$$

Each worker is assigned a `thread_index` and is responsible for finding `max_char` for each of the lines that *begin* in the section starting at

$$\texttt{batch}[(n \cdot \texttt{bytes\_per\_thread})]$$
$$\text{and is up to and including}$$
$$\texttt{batch}[\min(\texttt{batch\_size}, \quad (n+1) \cdot \texttt{bytes\_per\_thread})]$$

Each worker returns an array containing (in order) the value of `max_char` for each of the lines it is responsible for. The main thread can iterate over each thread and each `max_char` to retrieve the values in order.

For this to work we need to maintain a couple of invariants:

1. The start of the batch is always the start of a string. Besides the 0^{th} worker, all workers start at the beginning of their section but only compute `max_char` for lines after the first newline in their section (where the thread before them is responsible for that "leakage"). We require 1. so that the 0^{th} thread always has a place to start. Note that the last thread has to stop early if the last string continues into the next batch. In this case, to maintain 1. the main thread has to move those residual characters to the front of the buffer before filling it back up to make the next batch.
2. Each section needs to contain at least one newline. We rely on a newline in the threads partition to determine where its responsibility starts and another newline in the next partition to determine where its responsibility ends. In order to guarantee 2. we must have that

   $$\texttt{bytes\_per\_thread} \leq \texttt{batch\_size}$$

   which is true if and only if

   $$\texttt{max\_line\_length} \cdot \texttt{num\_workers} \leq \texttt{batch\_size}$$

   which we therefore have to maintain.

Versions Contrasted

pthreads

Under the pthreads paradigm the main() thread spawns each of worker threads which execute thread_routine(). Main passes each thread a pointer to the batch buffer and each thread returns a pointer to an array of their `max_char`'s.

MPI

Under the MPI paradigm we can not pass message by sharing memory so we must share memory by passing messages. Which is a fancy way to say that we have to copy all of the data between threads. In this way, the biggest difference in performance should be that the MPI code has to copy the batch buffer to each thread and the array of `max_char`'s from each thread.

OpenMP

`TODO`